

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I M.Niharika(192365019) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Niharika

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Set 4

1. Smart Home Automation System

A home automation company is developing an intelligent control system that automatically manages lighting, temperature, and security devices. The compiler embedded in the controller processes declarations, assignment statements, and Boolean expressions to activate devices based on sensor inputs and user preferences. During real-time execution, inefficient intermediate code generation caused delays in device response and security alerts. The development team plans to redesign the compiler to improve automation efficiency and reliability.

Tasks:

- (i) Design intermediate code for smart device control and sensor monitoring logic. (K3)
- (ii) Explain how Boolean expressions support automated decision-making in smart homes. (K4)
- (iii) Analyze the role of compiler optimization in improving home automation systems. (K5)

Answers

(i) *Design intermediate code for smart device control and sensor monitoring logic. (K3)*

Assume:

temp = room temperature

motion = motion sensor

dark = light sensor

security = security status

Three-address code (TAC):

1. *IF temp > 30 GOTO 4*
2. *GOTO 6*
3. *GOTO 6*
4. *CALL Turn_ON_Fan()*
5. *GOTO 7*
6. *CALL Turn_OFF_Fan()*

7. *IF dark == TRUE GOTO 10*

8. *GOTO 12*

10. *CALL Turn_ON_Light()*

11. *GOTO 13*

12. *CALL Turn_OFF_Light()*

13. *IF motion == TRUE AND security == TRUE GOTO 16*

14. *GOTO 18*

16. *CALL Activate_Alarm()*

17. *CALL Send_Security_Alert()*

18. *END*

This intermediate code converts sensor conditions into simple conditional jumps and procedure calls. It allows the compiler to translate the logic efficiently into machine instructions.

(ii) Explain how Boolean expressions support automated decision-making in smart homes. (K4)

Boolean expressions produce either TRUE or FALSE and are essential for making automatic decisions.

Examples:

temp > 30

motion == TRUE

dark == TRUE

motion == TRUE AND security == TRUE

For example:

IF temp > 30 AND humidity > 70

Turn_ON_AC()

The Boolean operators AND, OR, and NOT allow multiple sensor conditions to be combined.

AND – all conditions must be true.

OR – at least one condition must be true.

NOT – reverses a condition.

Thus, Boolean expressions allow the smart-home controller to automatically control lights, fans, air conditioners, alarms, and security systems without continuous human intervention.

(iii) Analyze the role of compiler optimization in improving home automation systems. (K5)

Compiler optimization improves the efficiency of smart-home software by reducing execution time and resource consumption.

Important optimizations include:

Constant folding – evaluates constant expressions during compilation.

Dead-code elimination – removes instructions that are never executed.

Common subexpression elimination – avoids calculating the same expression repeatedly.

Peephole optimization – replaces inefficient instruction sequences with shorter ones.

Control-flow optimization – reduces unnecessary jumps.

Code motion – moves calculations outside frequently repeated sections.

For a smart-home controller, optimization results in:

Faster sensor response.

Faster security-alert generation.

Lower CPU usage.

Lower power consumption.

Better real-time performance.

More reliable automation.

Therefore, compiler optimization is particularly important because even small delays can affect security and emergency responses.

2. Online Examination Management System

An educational technology company is building an online examination platform that evaluates students automatically and manages exam scheduling. The compiler processes declarations, assignment statements, case statements, and procedure calls for student authentication and result generation. During large-scale online examinations, inefficient intermediate code execution caused delays in question loading and score processing. The organization plans to optimize the compiler to improve scalability and performance.

Tasks:

- (i) Construct intermediate code for student authentication and evaluation logic. (K3)
- (ii) Discuss how case statements help manage different examination categories. (K4)
- (iii) Evaluate the importance of compiler optimization in large-scale online examination systems. (K5)

Answers

- (i) *Construct intermediate code for student authentication and evaluation logic.* (K3)

Assume:

username

password

valid

marks

Result

Three-address code:

1. *CALL Authenticate(username, password)*
2. *IF valid == TRUE GOTO 5*
3. *CALL Display("Invalid Login")*
4. *GOTO 20*
5. *CALL Load_Questions()*
6. *marks = 0*
7. *IF answer == correct_answer GOTO 10*
8. *GOTO 12*

10. $marks = marks + 1$

11. GOTO 13

12. $marks = marks + 0$

13. IF $marks \geq 40$ GOTO 16

14. $result = "FAIL"$

15. GOTO 18

16. $result = "PASS"$

18. CALL Store_Result(student_id, marks, result)

19. CALL Display_Result(marks, result)

20. END

The code separates authentication, examination processing, evaluation, and result generation.

(ii) Discuss how case statements help manage different examination categories. (K4)

A case statement is useful when a system needs to select one operation from many alternatives.

For example:

```
switch(exam_type)
{
    case 1:
        conductInternalExam();
        break;

    case 2:
        conductSemesterExam();
        break;

    case 3:
        conductPracticalExam();
        break;

    case 4:
        conductEntranceExam();
        break;
}
```

The compiler can convert the case statement into intermediate code using conditional jumps or a jump table.

Advantages:

Makes multiple choices easier to manage.

Improves program readability.

Avoids long sequences of nested if-else statements.

Allows efficient selection of examination rules.

Makes it easier to add new examination categories.

Thus, case statements provide structured control flow for examination management.

(iii) Evaluate the importance of compiler optimization in large-scale online examination systems. (K5)

Online examinations may involve thousands or millions of students simultaneously. Inefficient code can cause delays in authentication, question loading, answer processing, and result generation.

Compiler optimization can provide:

Faster authentication.

Faster question retrieval.

Efficient answer evaluation.

Reduced CPU and memory usage.

Better server scalability.

Lower response time.

Improved reliability during peak loads.

For example, removing redundant calculations from result processing can reduce the processing time for thousands of submissions.

Therefore, compiler optimization is critical for maintaining high throughput, low latency, scalability, and reliability in large-scale online examination platforms.

3. Intelligent Weather Forecasting System

A meteorological department is developing an intelligent weather forecasting application that analyzes environmental data in real time. The compiler processes assignment statements and Boolean expressions to monitor temperature, humidity, and rainfall conditions. During severe weather conditions, improper backpatching delayed alert generation and emergency notifications. The software engineers aim to redesign the compiler to improve prediction accuracy and system responsiveness.

Tasks:

- (i) Generate intermediate code for weather monitoring and alert generation logic. (K3)*
- (ii) Explain the role of backpatching in handling emergency weather alerts. (K4)*
- (iii) Analyze how compiler optimization enhances real-time forecasting applications. (K5)*

Answers

(i) Generate intermediate code for weather monitoring and alert generation logic. (K3)

Assume:

temperature

humidity

rainfall

alert

Three-address code:

- 1. IF temperature > 40 GOTO 10*
- 2. IF humidity > 85 GOTO 10*
- 3. IF rainfall > 100 GOTO 10*
- 4. alert = 0*
- 5. GOTO 15*

- 10. alert = 1*
- 11. CALL Generate_Weather_Alert()*
- 12. CALL Send_Emergency_Notification()*

15. *CALL Update_Weather_Status()*

16. *END*

The intermediate representation uses conditional branches and procedure calls to generate alerts when dangerous environmental conditions are detected.

(ii) Explain the role of backpatching in handling emergency weather alerts. (K4)
Backpatching is a compiler technique used to fill in the target addresses of jumps when those addresses are not known during the initial generation of intermediate code.

Consider:

if (temperature > 40 && rainfall > 100)

Generate_Alert();

else

Continue_Monitoring();

Initially, the compiler may generate:

if temperature > 40 goto _

goto _

if rainfall > 100 goto _

goto _

The blank targets are filled later using backpatching.

For Boolean expressions, backpatching maintains lists such as:

truelist – locations that should jump when the expression is true.

falselist – locations that should jump when the expression is false.

nextlist – locations for the next statement.

Correct backpatching ensures that an emergency alert is reached immediately when dangerous conditions become true. Incorrect targets could cause an alert to be delayed or skipped.

(iii) Analyze how compiler optimization enhances real-time forecasting applications. (K5)

Weather forecasting applications continuously process large volumes of sensor and environmental data.

Compiler optimization can:

Reduce computation time.

Remove unnecessary calculations.

Optimize conditional branches.

Reduce memory accesses.

Improve loop execution.

Reduce CPU usage.

Improve response time for emergency alerts.

For example, if the same temperature conversion is performed repeatedly, common-subexpression elimination can prevent unnecessary recalculation.

Optimization is especially important during severe weather because the system must process sensor information and generate warnings with minimal delay.

Therefore, compiler optimization improves speed, responsiveness, resource efficiency, and reliability in real-time forecasting systems.

4. Automated Library Management System

A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.

Tasks:

(i) Develop intermediate code for book issue and return operations. (K3)

(ii) Explain how procedure calls improve modularity in library management systems. (K4)

(iii) Evaluate the significance of compiler optimization in educational information systems. (K5)

Answers

(i) Develop intermediate code for book issue and return operations. (K3)

Assume:

member_id

book_id

available

fine

Intermediate code for book issue:

1. *CALL Verify_Member(member_id)*
2. *IF member_valid == TRUE GOTO 5*
3. *CALL Display("Invalid Member")*
4. *GOTO 20*

5. *CALL Check_Book(book_id)*
6. *IF available == TRUE GOTO 9*
7. *CALL Display("Book Not Available")*
8. *GOTO 20*

9. *CALL Issue_Book(member_id, book_id)*
10. *CALL Update_Catalog(book_id)*
11. *CALL Display("Book Issued")*
12. *GOTO 20*

Intermediate code for return:

13. *CALL Calculate_Fine(member_id, book_id)*
14. *IF fine > 0 GOTO 17*
15. *CALL Return_Book(book_id)*
16. *GOTO 19*

17. *CALL Collect_Fine(member_id, fine)*
18. *CALL Return_Book(book_id)*

19. *CALL Update_Catalog(book_id)*
20. *END*

This intermediate representation divides library operations into verification, availability checking, transaction processing, and catalog updates.

(ii) Explain how procedure calls improve modularity in library management systems. (K4)

Procedure calls divide a large system into smaller reusable functions.

For example:

Verify_Member()

Check_Book()

Issue_Book()

Return_Book()

Calculate_Fine()

Update_Catalog()

Each procedure performs a specific operation.

Benefits include:

Modularity – each operation is separated.

Reusability – the same procedure can be called from different screens.

Maintainability – changes can be made to one procedure.

Debugging – errors are easier to locate.

Code reduction – repeated code is avoided.

Scalability – new library features can be added easily.

Thus, procedure calls provide a clean and organized structure for library management software.

(iii) Evaluate the significance of compiler optimization in educational information systems. (K5)

Educational systems handle large amounts of data such as student records, library transactions, examinations, attendance, and fees.

Compiler optimization provides:

Faster transaction processing.

Reduced memory usage.

Faster database-related application logic.

Better server performance.

Improved response time during peak periods.

Reduced computational cost.

Better scalability.

During examinations, many students may simultaneously access educational systems. Optimized code helps prevent slowdowns and improves service availability.

Therefore, compiler optimization is important for achieving efficient, scalable, reliable, and responsive educational information systems.

5. Industrial Water Quality Monitoring System

An environmental engineering company is developing a water quality monitoring system that continuously checks pH levels, contamination, and chemical composition in industrial plants. The compiler in the embedded controller evaluates Boolean expressions, assignment statements, and case statements for automated monitoring and alert generation. During critical contamination events, inefficient intermediate code execution delayed warning notifications and corrective actions. The company aims to redesign the compiler to improve reliability and response time.

Tasks:

- (i)** Design intermediate code for contamination detection and alert handling logic. (K3)
- (ii)** Explain how case statements efficiently manage different water quality conditions. (K4)
- (iii)** Analyze the importance of compiler optimization in industrial environmental monitoring systems. (K5)

Answers

- (i)** Design intermediate code for contamination detection and alert handling logic. (K3)

Assume:

pH

contamination

chemical_level

status

Three-address code:

1. IF pH < 6.5 GOTO 10
2. IF pH > 8.5 GOTO 10
3. IF contamination > LIMIT GOTO 10
4. IF chemical_level > SAFE_LIMIT GOTO 10
5. status = "SAFE"
6. CALL Continue_Monitoring()
7. GOTO 20

10. status = "DANGER"
11. CALL Activate_Alarm()
12. CALL Send_Warning()
13. CALL Start_Corrective_Action()

20. CALL Store_Water_Status(status)
21. END

The intermediate code converts sensor conditions into conditional jumps and procedure calls for monitoring and emergency response.

(ii) Explain how case statements efficiently manage different water quality conditions. (K4)

Case statements are suitable when water quality can be classified into several predefined conditions.

Example:

```
switch(status)
{
    case 1:
        Display("Safe");
        break;

    case 2:
        Display("Warning");
        Activate_Alarm();
        break;

    case 3:
        Display("Critical");
```

```
    Activate_Alarm();  
    Start_Corrective_Action();  
    break;  
}
```

The compiler can implement these alternatives using conditional branches or a jump table.

Advantages:

Clear representation of different water conditions.

Faster selection between known cases.

Easier maintenance.

Avoids complicated nested if-else structures.

Makes alert-handling logic easier to understand.

For industrial monitoring, different responses can therefore be assigned to safe, warning, and critical conditions.

(iii) Analyze the importance of compiler optimization in industrial environmental monitoring systems. (K5)

Industrial monitoring systems often operate continuously and may need to respond immediately to dangerous conditions.

Compiler optimization is important because it can:

Reduce sensor-processing time.

Generate faster alert responses.

Reduce CPU and memory usage.

Eliminate redundant computations.

Optimize conditional branches.

Improve embedded-controller performance.

Reduce power consumption.

Increase system reliability.

For example, if contamination exceeds a critical limit, optimized code can reduce the time between sensor detection and activation of alarms or corrective equipment.

Therefore, compiler optimization is essential for real-time response, safety, reliability, resource efficiency, and continuous operation of industrial environmental monitoring systems.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

